

L04 · AXI 外壳与启动流程

作业说明 —— 任务拆解、验收标准与调试方法

对应上游仓库: [google/coralnpu](https://github.com/google/coralnpu) (RTL、文档、测试程序都来自这里)

学习方式: 读规格 → 写 RTL → `./learn check` 对拍 → 修复

文档版本: 2026-09-16 · 生成命令: `./Learn pdf`

目录

L04 作业说明

0. 交付物
1. 接口契约
2. 分步完成建议
3. 调试方法
4. 常见坑
5. 提交要求

L04 作业说明

0. 交付物

改两个文件（核心、ITCM、测试平台由课程提供，不要改）：

文件	TODO	内容
rtl/axi_lite_slave.sv	5 个	AXI4-Lite 从接口：AW/W/B 写通道、AR/R 读通道
rtl/axi_boot_shell.sv	6 个	地址译码、ITCM、三个控制寄存器、启动控制、核心侧路由、AXI 主状态机

课程提供：

文件	作用
tests/core_l04.sv	L03b 的核心 + i_pc_start（复位装载 PC）+ o_fault（异常且 mtvec=0）
tests/tcm.sv	ITCM：1 个主机写口 + 3 个组合读口（主机读、取指读、数据读）
tests/tb_axi.sv	主机 BFM（5 步启动）、系统存储器模型（DTCM 在 m_axi 后面）、trace/DTCM 采集
tests/link/learn_tcm_0x100.ld	把程序链接到 0x100，用来验证 PC_START 真的生效

验收：

```
./learn check L04          # 期望 10/10（5 个程序 × 2 种总线延迟）
./learn check L04 --only 10 # 只跑 5 步启动流程
./learn check L04 --only 12 # 只跑"没有处理程序的异常" → STATUS.FAULT"
./learn check L04 --ref     # 参考实现
```

1. 接口契约

这些端口为什么叫这个名字、每个 TODO 对应哪条验收、信号逐拍怎么变：见《基础知识》第 4 章「从协议到代码：信号名、端口与骨架 TODO 的对照」。

1.1 axi_lite_slave

```
module axi_lite_slave (
    input logic      clk, rst,
    // AXI4-Lite 从接口 (主机侧)
    input logic [31:0] s_awaddr, input logic s_awvalid, output logic s_awready,
    input logic [31:0] s_wdata, input logic [3:0] s_wstrb, input logic s_wvalid,
    output logic s_wready, output logic [1:0] s_bresp, output logic s_bvalid,
    input logic s_bready,
    input logic [31:0] s_araddr, input logic s_arvalid, output logic s_arready,
    output logic [31:0] s_rdata, output logic [1:0] s_rresp, output logic s_rvalid,
    input logic s_rready,
    // 本地端口 (交给外壳译码)
    output logic [31:0] o_wr_addr, o_wr_data, output logic [3:0] o_wr_strb,
    output logic o_wr_en, input logic [1:0] i_wr_resp,
    output logic [31:0] o_rd_addr, input logic [31:0] i_rd_data,
    input logic [1:0] i_rd_resp
);
```

约定：

1. **o_wr_en** 是**单拍脉冲**：外壳在这一拍拿着 **o_wr_addr/o_wr_data/o_wr_strb** 落盘；
2. 本地读是**组合的**：**o_rd_addr** 稳定时 **i_rd_data / i_rd_resp** 就有效；
3. **s_bvalid / s_rvalid** 是**电平**，不是脉冲——主机可能晚几拍才 **ready**。

1.2 axi_boot_shell

地址映射：

地址	行为
0x0000_0000 – 0x0000_1FFF	主机写 → ITCM；主机读 → ITCM（都返回 OKAY）
0x0003_0000	RESET_CONTROL：bit0=RESET，bit1=CLOCK_GATE，复位值 0x3
0x0003_0004	PC_START：复位值 0
0x0003_0008	STATUS（只读）：bit0=HALTED，bit1=FAULT
其它	SLVERR

核心侧：

通路	行为
取指 i_imem_addr → o_imem_rdata	组合读 ITCM（核心没有停顿信号，必须当拍给数据）
数据命中 ITCM	组合读，一拍完成；写 ITCM 丢弃（ITCM 只读）
数据未命中 ITCM	走 AXI 主接口； o_dmem_ready 在 R/B 到达的那一拍拉高

2. 分步完成建议

每一步做完都能在 `./learn check L04` 的日志里看到对应的变化（对照表见基础知识 4.3 节）。

Step 1: 从接口写通道（让主机的第一笔写能完成）

先在 `axi_lite_slave.sv` 里做 AW/W 握手 + `o_wr_en` + B 响应。做到一半可以先只求“握手能过”，跑 `./learn check L04 --only 10`，你会看到主机日志里 `HOST step1_words=...` 出现（说明写通路通了），后面才开始报 PC_START / STATUS 的问题。

Step 2: 从接口读通道 —— 主机要用它读回 PC_START 和 STATUS。

Step 3: 地址译码 + ITCM —— `wr_hit_itcm` / `rd_hit_itcm` / `rd_hit_csr`，例化课程给的 `tcm.sv`，把读数据与响应码接出来。未映射地址必须 `SLVERR`。

Step 4: 三个控制寄存器 + 启动控制 —— 注意 `o_core_clk_gate = cg_bit`、`o_core_rst = rst | reset_bit`、`o_core_pc_start = pc_start_q`。

Step 5: 核心侧路由 —— 取指读 ITCM；数据命中 ITCM 就地服务；未命中交给总线。

Step 6: AXI 主状态机 —— 读 `AR → R`、写 `AW+W → B`，在“事务真正完成的那一拍”拉高 `o_dmem_ready`（读的时候同时给出 `o_dmem_rdata`）。

3. 调试方法

① 看主机日志。每次运行都会打印：

```
HOST probe_unmapped=0x00000000 resp=2      ← 你返回的响应码 (2 = SLVERR)
HOST status_before=0x00000000 resp=0
HOST step1_words=89 entry=0x00000100 first=0x00018117 resp=0
HOST step2_pc_start=0x00000100 resp=0
HOST step3_reset_control=0x00000001 resp=0
HOST step4_reset_control=0x00000000 resp=0
HOST halted=1 status=0x00000001
HOST axi_txns rd=1 wr=4
```

② 打开总线事务打印（测试平台支持 `+AXI_TRACE=1`）：

```
cd course/work/L04/10_boot_lat0
vvp ../sim.vvp +PROGRAM=program.hex +DATA=data.hex +ENTRY=256 +LATENCY=0 \
+TRACE=t.txt +DMEM=d.txt +AXI_TRACE=1 | head -30
```

③ 分离两个文件的问题：`--axi` / `--shell` 可以只替换其中一个，另一个用参考实现：

```
./learn check L04 --shell course/lessons/L04_axi_boot/refs/axi_boot_shell.sv --only 10
```

4. 常见坑

现象	原因
仿真卡住，日志出现”主机/总线卡住”	<code>s_awready</code> / <code>s_wready</code> / <code>s_arready</code> 一直没拉高，或者 B/R 响应发不出去
主机第一步就报 <code>itcm_write_err</code>	<code>wr_hit_itcm</code> 的地址范围写错（ITCM 是 <code>0x0 - 0x1FFF</code> ）
<code>step2_pc_start</code> 读回 0	<code>PC_START</code> 的读地址（ <code>CSR_BASE+4</code> ）或写逻辑不对
核心从 0 开始跑（trace 第一条 <code>pc=0</code> ）	<code>o_core_pc_start</code> 没接；或者复位时 <code>i_pc_start</code> 不是 <code>PC_START</code>
核心一条指令都不执行	第 3、4 步顺序错了：核心没见过有效的复位
<code>m_axi</code> 上没有事务	数据访问没走总线（ <code>d_hit_itcm</code> 判断范围太大，或 <code>d_go_axi</code> 没接）
DTCM 里的值少了一部分字节	<code>WSTRB</code> 没从 <code>i_dmem_wmask</code> 传下去（非对齐写靠它）
读回来的数据是上一次的	读通路用了”连续赋值 + 读数组的函数”（iverilog 不会重新求值），要改成 <code>always_comb</code>

5. 提交要求

- `./learn check L04` 全绿（10/10），且 L01~L03b 的检查器仍然全绿；
- 不要改课程提供的文件（`tests/` 下的核心、ITCM、测试平台、链接脚本）；
- 代码里保留必要的注释，说明”这条握手规则为什么这么写”比”这行代码干了什么”更有价值。